



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«МИРЭА – Российский технологический университет»
РТУ МИРЭА**

Институт Информационных Технологий

Кафедра Вычислительной техники

**ОТЧЕТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ
№1**

**«Преобразование алгебраического выражения в обратную
польскую запись на Python»**

**по дисциплине
«Теория формальных языков»**

Выполнил студент группы ИКБО-43-23

Кошечев М. И.

Принял ассистент

Цынгальев П.С.

Практическая работа
выполнена

«16» сентябрь 2024 г.

«Зачтено»

«__» _____ 2024 г.

Москва 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ЦЕЛЬ	4
2 ЗАДАНИЕ	5
2.1 Формулировка задания	5
2.2 Математическая модель	5
2.3 Тестирование программы	6
ЗАКЛЮЧЕНИЕ	7
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	8
ПРИЛОЖЕНИЯ	9

ВВЕДЕНИЕ

Изучение теории формальных языков представляет собой значимый элемент образовательной программы по направлению программной инженерии. Эта область, являющаяся ключевой частью компьютерных наук и теоретической информатики, сосредотачивается на исследовании структуры и свойств формальных языков, а также на моделировании их обработки. В условиях современного общества, где возрастают объемы данных и потребность в эффективных методах их хранения, передачи и обработки, понимание основ формальных языков приобретает особую значимость.

Формальные языки находят применение в различных сферах: от описания синтаксиса языков программирования до разработки сетевых протоколов и создания компиляторов. С их помощью удастся формализовать взаимодействие как между человеком и компьютером, так и между компонентами программного обеспечения. В центре изучения теории формальных языков находятся такие понятия, как грамматики, автоматы и алгебраические структуры, которые позволяют описывать синтаксис и семантику языков.

1 ЦЕЛЬ

Преобразование алгебраического выражения в обратную польскую запись на Python.

2 ЗАДАНИЕ

2.1 Формулировка задания

На выбранном языке программирования реализовать преобразование простого алгебраического выражения в обратную польскую запись с использованием стека и простейший калькулятор выражений в обратной польской записи с целыми числами.

Выражение для обработки: $15 + 8 * (6 - 14/7) = 47,0$

2.2 Математическая модель

Разработка функции преобразования арифметического выражения в постфиксную запись и вычисления результата

Разработана функция для преобразования арифметического выражения в строковом формате в постфиксную запись (обратную польскую запись, ОПН). Исходное выражение передается в функцию в виде строки. В качестве первых шагов создается строковая переменная `data`, которая содержит арифметическое выражение, а также определяется список допустимых символов `white_list`, включающий цифры, операторы (+, -, *, /) и скобки. Для определения приоритета операторов используется словарь `operators`, который помогает различать операторы с разным уровнем приоритета.

В процессе работы функции `to_the_list` исходное арифметическое выражение преобразуется в последовательность токенов, состоящих из чисел, операторов и скобок. Каждый токен обрабатывается следующим образом:

- Если токен является числом, он добавляется в итоговый список.
- Если токен представляет собой оператор, то приоритет этого оператора сравнивается с приоритетом оператора на вершине стека с помощью функции `weight_checker`. Если приоритет текущего оператора меньше или равен приоритету оператора на вершине стека, то последний оператор переносится в итоговый список, и текущий оператор помещается в стек.
- Если токен является открывающей скобкой, она помещается в стек.
- Если токен является закрывающей скобкой, операторы из стека переносятся в итоговый список до тех пор, пока не будет найдена соответствующая открывающая скобка.

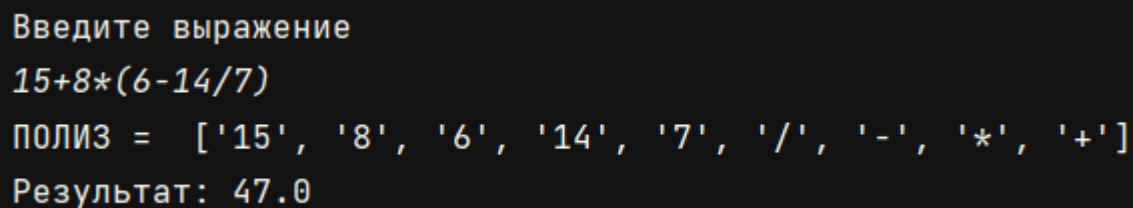
После обработки всех токенов оставшиеся операторы в стеке перемещаются в итоговый список, который представляет выражение в постфиксной нотации. Эта постфиксная запись затем передается в функцию для вычисления результата.

В функции `find_answer` вычисление значения постфиксного выражения осуществляется следующим образом: для каждого элемента постфиксной записи, если элемент является числом, он добавляется в стек. В случае оператора из стека извлекаются два последних операнда, над которыми выполняется соответствующая операция (сложение, вычитание, умножение или деление). Результат операции возвращается обратно в стек. В конце выполнения алгоритма в стеке остается единственное число, которое является результатом вычисления выражения.

Кроме того, разработана функция для проверки строки на наличие недопустимых символов, которая исключает любые символы, не входящие в набор цифр, операторов (+, -, *, /), скобок и пробелов, обеспечивая корректность ввода.

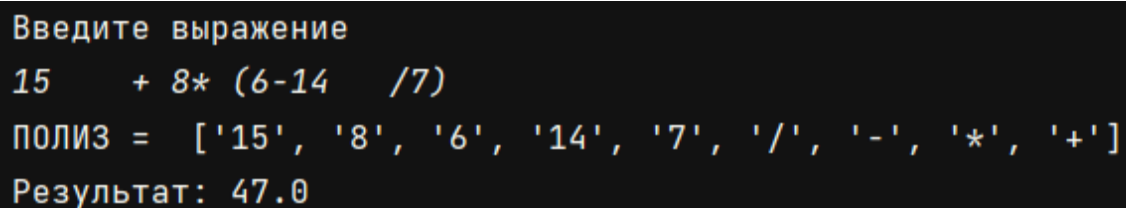
2.3 Тестирование программы

Проведём тестирование для данной программы на разных значениях. Результаты продемонстрируем на рисунке 1, 2.



```
Введите выражение
15+8*(6-14/7)
ПОЛИЗ = ['15', '8', '6', '14', '7', '/', '-', '*', '+']
Результат: 47.0
```

Рисунок 1 - Первое тестирование программы



```
Введите выражение
15 + 8* (6-14 /7)
ПОЛИЗ = ['15', '8', '6', '14', '7', '/', '-', '*', '+']
Результат: 47.0
```

Рисунок 2 - Второе тестирование программы

ЗАКЛЮЧЕНИЕ

В процессе выполнения практической работы были достигнуты следующие результаты:

Выполнено преобразование алгебраического выражения в обратную польскую запись с использованием Python;

Реализована программа, выполняющая преобразование алгебраического выражения в обратную польскую запись на Python;

Проведено тестирование программы, и результаты представлены в отчете.

Таким образом, поставленная цель работы — преобразование алгебраического выражения в обратную польскую запись на Python — успешно достигнута.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Малявко А.А. Формальные языки и компиляторы: учебное пособие для вузов. - М., Изд-во Юрайт, 2019.
2. Свердлов С.З. Языки программирования и методы трансляции : учебное пособие для вузов - СПб., 2007, Id=65534.
3. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов: учеб. пособие. - СПб.: БХВ-Петербург, 2005, Id=64347.

ПРИЛОЖЕНИЯ

Приложение А – код на языке Python

ПРИЛОЖЕНИЕ А

Код на языке Python

Листинг А.1 – main.py

```
white_list = ['1', '2', '3', '4', '5', '6', '7', '8', '9', '0',
              '*', '+', '-', '/', '(', ')']
operators = ['*', '+', '-', '/']

def to_the_list(data):
    input_data = list(data)
    input_data = [item for item in input_data if item.strip()]
    new_input_data = []
    number_sequence = ''
    for token in input_data:
        if token.isdigit():
            number_sequence += token
        elif token in white_list:
            if number_sequence:
                new_input_data.append(number_sequence)
                number_sequence = ''
            new_input_data.append(token)
    if number_sequence:
        new_input_data.append(number_sequence)
    if len(new_input_data) > 0:
        poliz_result = poliz(new_input_data)
        find_answer(poliz_result)
    else:
        print("Error")

def weight_checker(weight):
    if weight in ('+', '-'):
        return 1
    if weight in ('*', '/'):
        return 2
    return 0

def poliz(data):
    output = []
    stack = []
    for token in data:
        if token.isdigit():
            output.append(token)
        elif token in operators:
            while (stack and weight_checker(stack[-1]) >=
weight_checker(token)):
                output.append(stack.pop())
```

```
        stack.append(token)
    elif token == '(':
        stack.append(token)
    elif token == ')':
        while stack and stack[-1] != '(':
            output.append(stack.pop())
        if stack and stack[-1] == '(':
            stack.pop()
    while stack:
        output.append(stack.pop())
    print("ПОЛИЗ = ", output)
    return output

def operation(operand1, operand2, operation):
    if operation == '+':
        return str(float(operand1) + float(operand2))
    elif operation == '-':
        return str(float(operand1) - float(operand2))
    elif operation == '*':
        return str(float(operand1) * float(operand2))
    elif operation == '/':
        return str(float(operand1) / float(operand2))

def find_answer(poliz_result):
    stack = []
    for ducking_data in poliz_result:
        if ducking_data.isdigit():
            stack.append(ducking_data)
        elif ducking_data in operators:
            operand2 = stack.pop()
            operand1 = stack.pop()
            result = operation(operand1, operand2, ducking_data)
            stack.append(result)
    print("Результат:", stack.pop())

def main():
    print("Введите выражение")
    input_data = str(input())
    to_the_list(input_data)

main()
```